

NLP Basics & NLP Models — Summary of Learnings

Assignments 1-3: Text Cleaning, TF-IDF, and Word2Vec

1. Text Cleaning & Tokenization

Raw text data is messy — inconsistent casing, punctuation, and filler words all add noise that makes it harder for a model to find meaningful patterns. Working through this assignment on 25 movie reviews clarified why each cleaning step exists and what it actually changes in the data.

- **Lowercasing** ensures "Great" and "great" are counted as the same word, preventing the vocabulary from splitting unnecessarily by capitalization.
- **Removing punctuation** strips symbols that add no value at the word level and would otherwise create near-duplicate tokens like "movie" and "movie!".
- **Tokenization** converts a full sentence string into a list of individual words, the format almost every downstream NLP tool expects.
- **Removing stopwords** (e.g. "the", "is", "a") filters out very frequent but low-information words, leaving behind the words that actually characterize a review.

Key takeaway: cleaning is not one step but a pipeline of small, well-justified transformations, each removing a specific type of noise before the text is ready for numeric representation.

2. TF-IDF Implementation

TF-IDF converts cleaned text into numeric vectors by scoring each word in each document. Using scikit-learn's TfidfVectorizer on the cleaned reviews produced a matrix of shape (25 documents x number of unique words), where most entries are zero since any single review only uses a small fraction of the total vocabulary.

- A **high TF-IDF score** means a word appears often in one document but rarely across the rest of the dataset — making it a strong signal of what that document is specifically about.
- **IDF (Inverse Document Frequency)** is essential because term frequency alone would over-value common-but-uninformative words (like "movie" appearing in nearly every review). IDF down-weights words that show up everywhere, letting genuinely distinctive words rise to the top.

Key takeaway: TF-IDF is a simple, interpretable way to turn text into numbers, and it works well even on small datasets, but it has no notion of word meaning — it is purely frequency-based.

3. Word2Vec Embeddings

Training a small Word2Vec model with gensim on the same cleaned reviews produced a 50-dimensional dense vector for every word in the vocabulary, learned from the words that tend to appear near each other. With only 25 short reviews, the model's vocabulary and training signal are limited, so the similar-words results are best treated as a demonstration of the mechanics rather than production-quality embeddings.

TF-IDF vs Word2Vec — key differences:

- TF-IDF produces a single importance score per word per document; Word2Vec produces a dense vector per word that is independent of any specific document.

- TF-IDF has no concept of meaning; Word2Vec captures semantic similarity, so related words end up with similar vectors.
- TF-IDF vectors are as long as the vocabulary and mostly zeros (sparse); Word2Vec vectors have a small, fixed size regardless of vocabulary (dense).
- TF-IDF scores are tied to a specific document; Word2Vec vectors represent a word's general meaning across all the training text.
- TF-IDF works reasonably well on small datasets; Word2Vec needs much more text to learn reliable relationships.
- TF-IDF is commonly used for search and document similarity; Word2Vec is typically used as an input feature for deeper NLP models.

Overall Learnings

Working through all three assignments in sequence showed a clear progression: raw text must first be cleaned and tokenized before it can be represented numerically, and there are multiple valid ways to do that representation depending on the goal. TF-IDF is fast, interpretable, and frequency-based, making it a good fit for search and ranking tasks. Word2Vec is more computationally involved and needs more data, but captures meaning and context in a way TF-IDF cannot, making it more suitable as a foundation for downstream deep learning NLP tasks.